v1.0.0

OpenDccCLib

Full NMRA DCC Protocol Stack in C

Runs on any processor. No OS. No heap. Just clean C.

What is DCC?

DCC (Digital Command Control) is the NMRA standard for controlling model railroad locomotives and accessories. A command station sends addressed speed, function, and programming commands over the track rails as a modulated square wave. Decoders in locomotives and accessories receive and act on those commands.

DCC supports short and long addresses, 128-step speed control, up to 68 function outputs, CV-based configuration, service mode programming, and bi-directional communication via RailCom.

What is OpenDccCLib?

The full NMRA DCC protocol stack, written in plain C.

> *"No dynamic memory. No OS. No external dependencies. If your chip has a C compiler and a timer peripheral, it can run OpenDccCLib."*

Key Features

Zero heap allocation — Every buffer is a static array sized at compile time. No malloc, no fragmentation, no surprises.

Command station + decoder in one library — Build a command station, a decoder, or both from the same source tree. Compile-time feature flags select what you need.

Full NMRA service mode — Direct, paged, register, and address mode programming. ACK detection with configurable thresholds.

RailCom support — Command station cutout timing with one-shot timer state machine. Decoder-side 4/8-encoded responses via GPIO bit-bang. Full datagram decoding.

OS-free by design — No RTOS required. A single run() call per main-loop tick drives the entire protocol engine.

Dependency injection — Hardware callbacks are function pointers wired at init time. Swap platforms by changing the driver layer — the protocol code is untouched.

Compliance-tested — 23 gTest suites covering bit encoding, packet scheduling, service mode state machines, RailCom, CV storage, and decoder packet parsing.

BSD 2-Clause license — Use in open-source or commercial products with no royalties and no strings.

Protocol Coverage

Module	Details
Bit Encoder	NMRA-compliant 58µs one-bits, 100µs zero-bits, preamble generation, single-buffered packet serialization from ISR (one active packet + packet_loaded flag)
Packet Encoder	Speed (14/28/128), functions (F0–F68), basic & extended accessories, consist, binary state, analog, CV access (POM)
Scheduler	Static slot array with priority (ESTOP > SPEED > FUNCTION > ACCESSORY > CV > IDLE), duplicate combining via (address, tag), NMRA auto-refresh
Service Mode	Direct, paged, register, and address mode. Configurable ACK threshold, pulse duration, and retry count
RailCom Cutout	One-shot timer five-state machine: DELAY 26µs, SETTLING 54µs, CH1 97µs, GAP 16µs, CH2 261µs (~454µs total). User-configurable timings, H-bridge tristate control
RailCom Decoder	4/8 datagram decoding from 250 kbaud UART. Channel 1 and Channel 2 assembly
Bit Decoder	Edge timing classification with 80µs threshold. NMRA-compliant stretched zero-bit support
Packet Decoder	Preamble detection (≥10 bits), byte assembly, XOR validation, address matching, instruction dispatch
CV Storage	Read/write through function pointers. Decoder lock (CV15/CV16). Factory reset via CV8
RailCom Encoder	4/8 bit-bang encoding for decoder-side RailCom responses
Failsafe	Timeout detection, enter/exit callbacks for safe motor stop

Demo Platforms, Tools & Getting Started

Two ready-to-run example projects ship with the library. Porting to a new platform means writing just two driver files — a timer/GPIO driver and optionally a UART driver — then everything else runs unchanged.

Example	Platform	Toolchain / IDE
Command Station	TI MSPM0G3507 LaunchPad	Code Composer Studio / Theia
Decoder	TI MSPM0G3507 LaunchPad	Code Composer Studio / Theia

Architecture Highlights

- Static buffer pools sized at compile time — no malloc, no fragmentation
- Hardware access via nullable function-pointer interfaces (DI pattern)
- Single-tick main loop — no threads, no mutexes on single-core MCUs

- Shared 58µs timer drives both main and service track encoders simultaneously
- Edge-buffered decoder ISR under 1µs — never overruns the 58µs half-bit period
- Compile-time role selection: command station, decoder, or both

Unit Test Coverage

Metric	Detail
Test suites	23 gTest suites
Test cases	~923 test cases
Source files	48 .c/.h files in src/dcc/
Source lines	~13,700 lines

23 gTest suites cover the full source tree (bit encoder, scheduler, packet encoder, service mode state machines, RailCom cutout, RailCom decoder, bit decoder, packet decoder, CV storage, RailCom encoder, config lifecycle, application API).

Getting Started

1. **Quick Start (Command Station)** — Import applications/ti_theia/mspm03507_launchpad/command_station/ into CCS, build, flash, and type POWER ON. DCC packets on the track in minutes.

2. **Quick Start (Decoder)** — Import applications/ti_theia/mspm03507_launchpad/decoder/ into CCS, build, flash, and connect to a DCC source. Decoded commands print over UART.

3. **New platform** — Copy an example project. Rewrite the driver files for your MCU's timer and GPIO. The entire protocol engine runs unchanged.

Documentation & Repository

Quick Start Guide (Command Station) — Step-by-step MSPM0G3507 walkthrough
Quick Start Guide (Decoder) — Step-by-step decoder walkthrough
Developer Guide (Command Station) — Config struct, scheduler, bit encoder, service mode, RailCom, porting
Developer Guide (Decoder) — Bit decoder, packet decoder, CV storage, callbacks, porting
API Reference — Doxygen-generated HTML

github.com/jimkueneman/OpenDccCLib — BSD 2-Clause License

> *Built for model railroaders, by a model railroader.*